📚 Clinic Management Project Overview

Brief Overview

This note covers **lightweight, file-based system**. It walks you through the folder hierarchy, Git setup steps, core Python modules for patient records, and utility functions for vitals and analytics.

Key Points

- Folder structure and cross-platform commands for creating project files.
- Git workflow basics: initializing, staging, committing, and clear commit messages.
- Data models and utility functions (patient IDs, blood-group validation, name cleaning).
- Roadmap for adding vitals calculations, analytics, and a menu-driven user interface.

📋 Project Overview

- Small clinic needs a **lightweight, file-based system** (no database, no internet).
- All data will be stored in plain files and managed via file-handling operations.

🗂️ Requirements

> The system must:

- Store **patient records**: name, gender, blood group, allergies, vitals.
- Automatically compute **BMI**, **blood-pressure stage**, and generate an overall health report per patient.
- Provide functions to **search**, **add**, and **log** patient information.
- Generate reports:
  - List of **high-risk patients**.
  - **Average age** of patients.
  - All **allergies** related to a particular patient.
- Display the clinic's **department hierarchy**.

📂 Folder Structure & Project Setup

Folder / File	Purpose
Project Healthcare Meditrack/	Root project directory (created on Desktop).
meditrack/	Contains all Python modules.
data/	Holds log files (patient.txt, patient_visit.txt).
main.py	Entry point; shows menu and demo.
README.md	Project documentation.
meditrack/__init__.py	Marks meditrack as a package.
meditrack/data.py	Data models and structures.
meditrack/utils.py	Utility functions.
meditrack/vitals.py	Vital-sign calculations (BMI, BP stage).
meditrack/patients.py	Patient-record management.
meditrack/analytics.py	Reporting and analytics functions.
meditrack/storage.py	File-handling logic.

Steps to Create the Structure

1. **Create root folder** on Desktop named Project Healthcare Meditrack.
2. Open **VS Code**, *File → Open Folder* → select the newly created folder.

3. Install **Git** (download from the official site, use default settings).
4. Open the **integrated terminal** in VS Code.

🛠️ Development Environment Setup

- **VS Code** is the IDE for all project work (data analytics, data science, Python).
- **Git** will track changes; a GitHub account can be created later.

Installing VS Code

- Search "VS Code" on Google, download from the official site, run the default installer.

Installing Git

- Download from the official Git website (Windows, macOS, Linux).
- Follow the default installation steps.
- If git is not recognized, add the Git binary to the system **PATH**:
    1. Search "environment variable" → **Edit system environment variables**.
    2. In **User variables**, edit **Path**, add the Git installation directory (e.g., C:\Program Files\Git\bin).

🔧 Initializing a Git Repository

```
git init
```

- Creates a hidden .git folder; the project is now under version control.

📁 Creating Files & Folders via Terminal

For macOS / Linux (using touch and mkdir):

```
mkdir -p meditrack data
touch main.py README.md
touch meditrack/__init__.py meditrack/data.py meditrack/utils.py \
      meditrack/vitals.py meditrack/patients.py meditrack/analytics.py \
      meditrack/storage.py
```

For Windows (using PowerShell New-Item):

```
mkdir meditrack, data
New-Item -Path . -Name main.py, README.md -ItemType File
New-Item -Path meditrack -Name __init__.py, data.py, utils.py, vitals.py, patients.py, analytics.py, storage.
```

- If commands fail, open **Git Bash** (installed with Git) and run the macOS/Linux commands there.

📊 Git Workflow Overview

Area	Description
Working area	Files you edit locally.
Staging area	Intermediate step; files added with git add.
Commit area	Permanent snapshot; created with git commit.

Common Commands

1. **Check status**

```
git status
```

2. **Stage all changes**

```
git add .
```

3. **Commit with message**

```
git commit -m "Created project files and folder structure"
```

4. **View commit history**

```
git log --oneline
```

- After committing, the terminal will show **"working tree clean"**, indicating no pending changes.

🧩 Adding and Committing Files

1. After creating files/folders, run git status → shows them as *untracked*.
2. Stage them: git add . → they move to the *staged* list.
3. Commit: git commit -m "Project files and folder structure created" → snapshot saved with a commit ID.

📚 Additional Notes

- Use **Git Bash** for all terminal commands on Windows to avoid PowerShell incompatibilities.
- Ensure you are inside the project root (Project Healthcare Meditrack) before running any Git or file-creation commands.
- The .git directory is hidden; it contains all version-control metadata.

All steps above reflect the exact instructions and information conveyed during the lecture.

🔧 Git Configuration & Common Issues

- **Set global user name**

```
git config --global user.name "Your Name"
```

If the name contains spaces, wrap it in quotation marks.

- **Set global user email**

```
git config --global user.email "you@example.com"
```

- **Why configure?**

> When committing, Git records the author's name and email. This information is essential in collaborative repositories so that each change can be attributed to the correct contributor.

- **Typical error** – *"Not a git repository"*
  - Occurs when commands are executed outside the project folder or in the wrong terminal.
  - **Solution:** Open **Git Bash** inside the project root (Project Healthcare Meditrack) and run the commands there.
- **Workflow recap**
  1. git status → view untracked/modified files.

2. git add . → move files to the **staging area**.
3. git commit -m "Message" → create a commit snapshot.

- **One-time setup**
  - The global user name/email need to be set **once per machine**.
  - When switching to a new laptop, repeat the git config commands.

📂 Data Structures for Patient Records

Patients List (in meditrack/data.py)

- **Structure:** patients → *list* of *dictionaries*; each dictionary represents one patient.

Key	Type	Reason
id	str	Unique identifier (format: PAT-1001-XXXX-XXXX).
name	str	Patient's full name.
dob	str (YYYY-MM-DD)	Fixed date format for reliable age calculations.
gender	str	Stored as a plain string (e.g., "Male", "Female").
blood_group	str	Must match one of the eight standard groups.
allergies	set	**Set** avoids duplicate entries for the same allergy.
vitals	dict (nested)	Holds height, weight, systolic, diastolic, heart rate, temperature.
visits	list of tuple	Each tuple records a single visit; immutability preserves historic data.

Rationale for chosen containers

- **Set for allergies** – Guarantees *unique* allergy entries; adding the same allergy twice has no effect.
- **Frozen set for valid blood groups** – Blood groups are immutable; a frozenset prevents accidental modification.
- **List of tuples for visits** – Allows multiple visits while keeping each visit record immutable, preserving audit trails.

🛠️ Utility Functions (meditrack/utils.py)

1. Valid Blood Groups

```
VALID_BLOOD_GROUPS = frozenset(
    {"A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"}
)
```

> *A frozen set is used because the eight blood groups are constant and should never be altered at runtime.*

2. clean_name(raw_name)

```
def clean_name(raw_name: str) -> str:
    """
    Normalize a name string:
    - Remove leading/trailing spaces.
    - Collapse multiple internal spaces to a single space.
    - Convert to title case.
    """
```

```
    cleaned = " ".join(raw_name.strip().split())
    return cleaned.title()
```

- **Steps:** strip() → split() → " ".join(...) → title().

3. is_valid_blood_group(blood_group)

```
def is_valid_blood_group(blood_group: str) -> bool:
    """Return True if the supplied blood group (case-insensitive) is in VALID_BLOOD_GROUPS."""
    return blood_group.strip().upper() in VALID_BLOOD_GROUPS
```

4. Patient ID Generation

```
import random

_ID_COUNTER = 1000  # module-level counter (private)

def generate_patient_id() -> str:
    """
    Produce a unique patient identifier:
    - Increment the global counter.
    - Append a random 4-digit suffix (1000-9999).
    - Format: 'PAT-{counter}-{suffix}'
    """
    global _ID_COUNTER
    _ID_COUNTER += 1
    suffix = random.randint(1000, 9999)
    return f"PAT-{_ID_COUNTER}-{suffix}"
```

- **Global keyword** ensures the counter persists across calls.
- The random suffix guarantees additional uniqueness beyond the sequential counter.

📊 Commit Best Practices

- **Separate concerns:**
  - When modifying only data.py, stage and commit that file alone.
  - When adding utility functions, create a distinct commit for utils.py.
- **Clear commit messages** (example):

```
git commit -m "Add initial patient records in data.py"
git commit -m "Implement VALID_BLOOD_GROUPS, clean_name, is_valid_blood_group, and generate_patient_i
```

- **Avoid mixing unrelated changes** in a single commit, as it obscures the purpose of each modification and hampers code review.

📦 Preparing for Future Work

- **Branching (to be introduced later):**
  - Each new feature (e.g., patient CRUD operations, analytics) will be developed on its own branch.
  - After completion, the branch will be merged, preserving a clean commit history.
- **Next steps:**
  1. Populate patients list with three realistic records (as demonstrated in the lecture).
  2. Use the utility functions to validate and clean user input when adding new patients.

3. Continue building higher-level modules (patients.py, analytics.py) while adhering to the commit discipline outlined above.

🪝 Git Commit Workflow Recap 📂

> **Definition:** *A Git commit records a snapshot of the current project state, moving changes from the staging area to the repository history.*

- After adding files with git add ., git status will list **"Changes to be committed"**.
- Execute the commit:

```
git commit -m "Initial commit with project structure"
```

- Verify the commit history with:

```
git log --oneline
```

- If git log shows **"no commits yet"**, ensure that git add was run on the correct files and that the commit command was executed without errors.

🗺️ Project Development Roadmap – Steps 3-7 🚀

Step	Focus	Key Tasks	Status
3	*Encode clinical rules*	Implement vital-sign calculations (BMI, BP stage) in meditrack/vitals.py.	Pending
4	*Patient-record management*	Complete CRUD functions in meditrack/patients.py (add, search, log visits).	Pending
5	*Analytics & reporting*	Build high-risk patient list, average age, allergy queries in meditrack/analytics.py.	Pending
6	*File-handling layer*	Finalise read/write logic for patient.txt and patient_visit.txt in meditrack/storage.py.	Pending
7	*User interface*	Wire up menu system in main.py to invoke the above modules; add clear-screen helper.	Pending

> **Note:** Steps 1 & 2 (data modeling and reusable utilities) have been completed as described in earlier sections.

📌 Pending Functions to Implement (Utilities)

- **calculate_age(dob)** – Compute patient age from date of birth using the datetime module.
- **next_appointment_slots(current_date, interval_days)** – Generate future appointment dates based on a configurable interval.
- **clear_screen()** – Platform-aware command to clear the terminal window.
- **valid_blood_group(blood_group)** – Return True only if the input matches one of the predefined groups (already defined in utils.py).

These utilities will be placed in meditrack/utils.py and referenced by later modules.

📅 Next Class Plan & Checklist ✅

1. Review **Git and GitHub concepts** (revision of staging, committing, and pushing).
2. Implement the **age calculation** and **appointment slot** helpers.
3. Begin coding **vitals.py**:
    - BMI formula: $\text{BMI} = \frac{\text{weight (kg)}}{(\text{height (m)})^2}$
    - BP stage mapping based on systolic/diastolic thresholds.
4. Incrementally test each new function, committing after every logical change.

> *The instructor emphasized that completing these items will likely require one or two additional sessions before the project is finished.*
